



ReportA

Sistema de Reportes Ciudadanos

v 1.0 — Centro Universitario de Tonalá

DOCUMENTACIÓN TÉCNICA COMPLETA

Arquitectura de Microservicios · Spring Boot · Astro · Docker

INTEGRANTES DEL EQUIPO

Corona Preciado Fernando

García de la Cruz Jesús Axel

Fierros García Carlos Samuel

Rodríguez Armas Juan Ramón

Mayo 2026

Tabla de Contenido

1. Descripción General del Sistema
2. Objetivos del Proyecto
3. Arquitectura del Sistema
4. Tecnologías y Stack Tecnológico
5. Servicios y Microservicios en Detalle
6. Base de Datos
7. Autenticación y Seguridad
8. Frontend — Astro (reporta-app)
9. Comunicación entre Servicios
10. Dockerización y Despliegue
11. Flujos Principales de Usuario
12. Endpoints API — Referencia Rápida
13. Variables de Entorno y Configuración
14. Guía de Instalación y Ejecución
15. Repositorio del Proyecto
16. Glosario de Términos

1. Descripción General del Sistema

ReportA es una plataforma de reportes ciudadanos orientada a Jalisco, México, que permite a los habitantes de un municipio documentar problemas de infraestructura urbana (baches, alumbrado deficiente, acumulación de basura, daños en banquetas, etc.) de forma sencilla, geolocalizando el incidente con un simple clic en un mapa interactivo. Las autoridades locales reciben y gestionan dichos reportes desde un panel administrativo dedicado.

El sistema está construido sobre una arquitectura de microservicios independientes, orquestados mediante Docker Compose y descubiertos dinámicamente a través de Eureka (Netflix OSS). Cada servicio tiene su propia base de datos, ciclo de vida y responsabilidad bien delimitada.

Propósito y alcance

- Brindar un canal digital oficial para que ciudadanos reporten problemas urbanos.
- Permitir a las autoridades municipales gestionar, priorizar y resolver incidencias.
- Mantener trazabilidad completa: fecha, ubicación GPS, descripción, imágenes e historial de estados.
- Escalar horizontalmente al tratarse de microservicios desacoplados.

Roles de usuario

Rol	Capacidades
usuario	Ciudadano. Puede crear, ver, editar y eliminar sus propios reportes. Requiere verificación de correo.
autoridad	Administrador municipal. Puede ver todos los reportes de su municipio, cambiar estados y agregar observaciones.

2. Objetivos del Proyecto

Objetivos funcionales

1. Registro, autenticación y verificación de usuarios vía correo electrónico.
2. Creación de reportes ciudadanos con geolocalización inversa (Google Maps API).
3. Gestión del ciclo de vida de un reporte: PENDIENTE → EN_PROCESO → RESUELTO / RECHAZADO.
4. Panel administrativo filtrado por municipio y estado del reporte.
5. Almacenamiento y servicio de imágenes adjuntas a cada reporte.

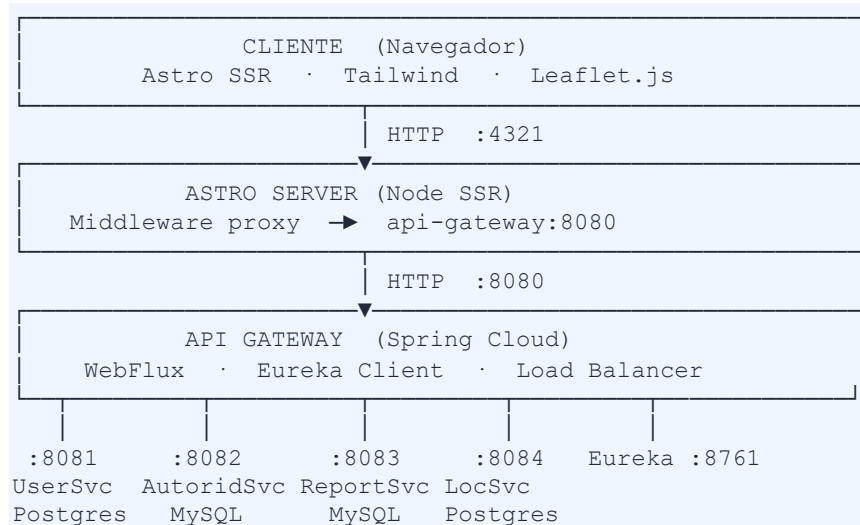
Objetivos técnicos

- Arquitectura de microservicios con independencia de despliegue por servicio.
- Descubrimiento de servicios dinámico mediante Eureka Server.
- Enrutamiento centralizado a través del API Gateway con load balancing.
- Seguridad stateless basada en JWT compartido entre servicios.
- Contenedorización completa con Docker y Docker Compose.

3. Arquitectura del Sistema

ReportA implementa un patrón de Microservicios con descubrimiento dinámico y gateway centralizado. Cada microservicio es autónomo, con su propia base de datos y Dockerfile, desplegado como un contenedor Docker independiente.

Diagrama conceptual de capas



Microservicios y puertos

Servicio	Puerto	Responsabilidad
eureka-server	:8761	Registro y descubrimiento de servicios (Netflix Eureka).
api-gateway	:8080	Puerta de entrada única. Enruta peticiones a los servicios vía Eureka lb://.
UserService	:8081	Gestión de usuarios, autenticación, emisión de JWT, verificación de correo.
autoridades-service	:8082	Alta de autoridades, cambio de rol de usuario, vinculación con municipio.

Servicio	Puerto	Responsabilidad
report-service	:8083	CRUD de reportes, subida de imágenes, cambio de estado, filtros por municipio.
location_service	:8084	Geocodificación inversa/directa vía Google Maps API, persistencia de ubicaciones.
reporta-app	:4321	Frontend SSR en Astro. Proxy interno hacia api-gateway:8080.

4. Tecnologías y Stack Tecnológico

Backend

Tecnología	Versión / Detalle
Lenguaje	Java 21 / 25 (según servicio)
Framework	Spring Boot 4.0.x
ORM	Spring Data JPA + Hibernate
Seguridad	Spring Security + JWT (jjwt 0.11.5)
Correo	Spring Mail (SMTP Gmail)
Descubrimiento	Spring Cloud Netflix Eureka Client / Server
Gateway	Spring Cloud Gateway WebFlux (lb:// load balancing)
HTTP reactivo	WebFlux (WebClient para comunicación entre servicios)
Build	Apache Maven 3.9.x / Maven Wrapper
Lombok	v1.18.38 (reducción de boilerplate)
Maps	Google Maps Services Java Client v2.2.0

Frontend

Tecnología	Detalle
Framework	Astro 6.3.x (SSR mode, adapter: @astrojs/node)
Runtime	Node.js >= 22.12.0
Estilos	Tailwind CSS v4 + CSS custom properties (design system propio)
Mapas	Leaflet.js 1.9.4 (integración CDN)
Tipografías	Syne (títulos) + IBM Plex Mono (código) — Google Fonts

Tecnología	Detalle
Lenguaje	TypeScript (tsconfig strict)
Bundler	Vite (incorporado en Astro)
Proxy SSR	Middleware Astro: reescribe /api/* → http://api-gateway:8080

Bases de datos

Instancia	Motor · Contenido
postgresql-user	PostgreSQL 13 · Usuarios, credenciales, tokens de verificación
mysql-report	MySQL 8 · Reportes ciudadanos e imágenes
postgresql-location	PostgreSQL 13 · Coordenadas y direcciones geocodificadas
mysql-autoridades	MySQL 8 · Autoridades y municipios de Jalisco (125 registros)

Infraestructura y DevOps

Aspecto	Detalle
Contenedores	Docker + Docker Compose (multi-stage builds)
Imagen base build	maven:4.0.0-rc-5-sapmachine-25 (stage 1)
Imagen base run	eclipse-temurin:25-jdk-alpine (stage 2 — mínima)
Frontend image	node:22-alpine (build) + node:22-alpine (run)
Healthchecks	mysqladmin ping / pg_isready en todos los contenedores DB
Volúmenes Docker	Persistencia de datos DB + uploads de imágenes

5. Servicios y Microservicios en Detalle

5.1 Eureka Server

Servidor de registro centralizado para todos los microservicios. Cada servicio se registra al inicio con su nombre de aplicación (spring.application.name) y el gateway los descubre usando lb://NOMBRE para balanceo de carga automático.

```
Puerto: 8761
URL Dashboard: http://localhost:8761
No registra consigo mismo (register-with-eureka=false)
```

5.2 API Gateway

Único punto de entrada para todos los clientes. Implementado con Spring Cloud Gateway WebFlux. Define rutas estáticas en application.yaml que mapean prefijos de path a servicios registrados en Eureka.

Rutas configuradas:

Prefijo de ruta	Servicio destino (Eureka)
/api/report/**	lb://report-service
/api/ubicación/**	lb://ubicacion
/api/autoridad/**	lb://autoridades-service
/api/users/**	lb://user-service
/api/auth/**	lb://user-service

5.3 UserService

Microservicio responsable de todo el ciclo de vida del usuario: registro, verificación de cuenta, autenticación y emisión de JWT. Es también el proveedor de identidad para el resto del sistema.

Entidades y lógica clave

- User: id, name, email (unique), password (BCrypt), tipoUser (usuario | autoridad), activo, tokenVerificacion.
- Al registrarse: se genera UUID como token de verificación y se envía por correo SMTP (Gmail).
- Al verificar: se activa la cuenta (/api/users/verify?token=...) y se limpia el token.
- Login: Spring AuthenticationManager valida credenciales; si cuenta activa, genera JWT HS256 con claims id y tipoUser.
- PATCH /api/users/cambiar-tipo/{email}: alterna el rol usuario <-> autoridad (llamado por autoridades-service).

5.4 autoridades-service

Gestiona el alta y actualización de perfiles de autoridad municipal. Al crear una autoridad, realiza una llamada HTTP reactiva (WebClient) al UserService para cambiar el rol del usuario a "autoridad" y obtener su id.

- POST /api/autoridad — crea autoridad: llama a UserService, valida municipio en BD, guarda.
- PUT /api/autoridad/{id} — actualiza teléfono y municipio.
- GET /api/autoridad/usuario/{idUserario} — retorna detalle de la autoridad con nombre del municipio.

NOTA: Este servicio carga 125 municipios de Jalisco en su base de datos mediante init.sql.

La tabla `municipios` es de solo lectura para el servicio; se gestiona únicamente vía SQL inicial.

5.5 report-service

Núcleo del sistema de reportes. Gestiona el CRUD completo de incidencias ciudadanas, almacenamiento de imágenes en disco y comunicación con `location_service` para filtrar por municipio.

Endpoint	Tipo	Descripción
POST <code>/api/report</code>	multipart/form-data	Crear reporte con imagen opcional (JWT requerido).
GET <code>/api/report/{id}</code>	—	Obtener detalle de un reporte por ID.
GET <code>/api/report/usuarios/{id}</code>	—	Listar reportes de un usuario.
GET <code>/api/report/municipio/{municipio}</code>	—	Reportes filtrados por municipio (vía <code>location_service</code>).
PUT <code>/api/report/{id}</code>	multipart/form-data	Actualizar descripción e imagen.
PUT <code>/api/report/actualizar/estatus</code>	JSON	Cambiar estado + observaciones (solo autoridad).
DELETE <code>/api/report/{id}</code>	—	Eliminar reporte.
GET <code>/api/report/images/{id}</code>	—	Servir imagen del reporte (público).

Estados de reporte (enum ordinal):

Estado	Descripción
PENDIENTE (0)	Estado inicial al crear el reporte. Aguarda atención de la autoridad.
EN_PROCESO (1)	La autoridad ha tomado acción y está gestionando la incidencia.
RESUELTO (2)	El problema reportado ha sido resuelto satisfactoriamente.
RECHAZADO (3)	El reporte fue descartado (duplicado, improcedente, etc.).

5.6 location_service

Servicio de geocodificación que actúa como intermediario entre la aplicación y la Google Maps API. Persiste cada ubicación consultada en PostgreSQL para evitar reconsultas y mantener historial.

- POST `/api/ubicacion/reverse?lat=&lng=` — geocodificación inversa: coordenadas → dirección completa + municipio. Persiste y retorna ID.

- GET /api/ubicacion/forward?query= — geocodificación directa: texto → coordenadas.
- GET /api/ubicacion/{municipio} — retorna lista de IDs de ubicaciones en ese municipio (usado por report-service).
- GET /api/ubicacion/id/{id} — detalle completo de una ubicación por su ID.

6. Base de Datos

Cada microservicio posee su propia instancia de base de datos, alineado con el principio de "database per service" de los microservicios. No existe ninguna base de datos compartida entre servicios.

Esquema: UserService (PostgreSQL)

Columna	Tipo / Restricción
id	BIGSERIAL PRIMARY KEY
name	VARCHAR NOT NULL
email	VARCHAR UNIQUE NOT NULL
password	VARCHAR NOT NULL (BCrypt hash)
tipo_usuario	ENUM (usuario autoridad) DEFAULT usuario
activo	BOOLEAN DEFAULT false
token_Verificacion	VARCHAR NULLABLE

Esquema: report-service (MySQL)

Columna	Tipo / Detalle
id_reportes	BIGINT AUTO_INCREMENT PRIMARY KEY
id_usuario	BIGINT NOT NULL (referencia lógica a UserService)
id_ubicacion	BIGINT NOT NULL (referencia lógica a location_service)
descripcion	TEXT
fecha_reporte	DATETIME NOT NULL (auto @PrePersist)
observaciones	TEXT NULLABLE
estado	TINYINT (0=PENDIENTE, 1=EN_PROCESO, 2=RESUELTO, 3=RECHAZADO)
nombre_imagen	VARCHAR NULLABLE (filename en disco)

Esquema: location_service (PostgreSQL)

Columna	Tipo / Detalle
id	BIGSERIAL PRIMARY KEY
latitud	DOUBLE PRECISION NOT NULL
longitud	DOUBLE PRECISION NOT NULL
direccion_completa	TEXT
calle / numero	VARCHAR
colonia	VARCHAR
codigo_postal	VARCHAR
municipio	VARCHAR NOT NULL
estado	VARCHAR NOT NULL
fecha_consulta	TIMESTAMP (auto @PrePersist)

Esquema: autoridades-service (MySQL)

Columna (tabla autoridades)	Tipo / Detalle
idAutoridades	INT AUTO_INCREMENT PRIMARY KEY
idUsuarios	INT NOT NULL (referencia lógica a UserService)
Telefono	VARCHAR(10) NOT NULL
municipio	INT (FK → municipios.id_municipio)

Columna (tabla municipios)	Tipo / Detalle
id_municipio	INT PRIMARY KEY
municipio	VARCHAR(255) — 125 registros de Jalisco

7. Autenticación y Seguridad

El sistema utiliza JWT (JSON Web Token) como mecanismo de autenticación stateless. El UserService emite el token; todos los demás servicios que requieren autenticación validan la firma localmente sin consultar al UserService.

Flujo de autenticación

11. El usuario envía email + password a POST /api/auth/login.
12. UserService verifica credenciales vía Spring AuthenticationManager + BCryptPasswordEncoder.

13. Comprueba que la cuenta esté activa (activo=true).
14. Genera JWT HS256 firmado con la clave en jwt.secret (Base64).
15. El token contiene: sub (email), id (user id), tipoUser (rol), iat, exp (8 horas).
16. El frontend almacena el JWT en una cookie "jwt" (accesible desde JS y SSR).
17. Cada request protegido incluye Authorization: Bearer <token>.
18. Los servicios backend validan firma y expiración localmente (JwtAuthFilter).

Estructura del token JWT

```
// Header
{ "alg": "HS256", "typ": "JWT" }

// Payload
{
  "sub": "usuario@correo.mx",
  "id": 42,
  "tipoUser": "usuario",
  "iat": 1716000000,
  "exp": 1716028800 // +8 horas
}
```

IMPORTANTE: La clave jwt.secret debe ser idéntica en UserService y report-service. Nunca debe exponerse en el repositorio. Usar variables de entorno en producción.

Verificación de cuenta por correo

Al registrarse, el usuario recibe un correo con enlace `http://<IP>:8080/api/users/verify?token=<UUID>`. Hasta que no verifica su cuenta, la autenticación es rechazada con 403 CuentaNoVerificadaException. El envío se realiza vía JavaMailSender con SMTP Gmail (TLS 587).

8. Frontend — reporta-app (Astro SSR)

La aplicación de usuario final está construida con Astro 6 en modo SSR (Server-Side Rendering), desplegada sobre Node.js mediante el adaptador @astrojs/node standalone. Esto permite lógica de servidor en los frontmatters de cada página (autenticación, fetch de datos, redirects) sin exponer el backend al navegador directamente.

Estructura de páginas y rutas

Ruta	Descripción
/	Landing page pública con hero, features y CTA.
/login	Formulario de inicio de sesión (POST SSR → UserService).
/register	Formulario de registro (POST SSR → UserService).

Ruta	Descripción
/logout	Elimina cookie JWT y redirige a /login.
/reports	Lista de reportes del usuario autenticado.
/reports/create	Formulario con mapa Leaflet para crear un reporte.
/reports/[id]	Detalle de un reporte con imagen, mapa popup y datos de ubicación.
/reports/edit/[id]	Edición de descripción e imagen de un reporte propio.
/admin/reports	Panel administrativo: filtros, tabla, cambio de estado, eliminación.

Sistema de diseño (Design System)

Variable CSS	Valor / Uso
--bg	#0D0E11 — Fondo principal (casi negro)
--surface	#14161C — Superficies de cards
--accent	#F59E0B — Color de acción y énfasis (ámbar)
--text	#E8EAF0 — Texto principal
--danger	#EF4444 — Errores y eliminación
--success	#22C55E — Éxito y resolución
--font-display	Syne (títulos y UI)
--font-mono	IBM Plex Mono (código y badges)

Middleware de proxy (SSR)

El archivo `src/middleware.ts` intercepta todas las peticiones que comiencen por `/api` y las reenvía directamente al `api-gateway:8080`, eliminando el header `origin` para evitar problemas CORS. El navegador nunca llama al backend directamente; siempre pasa a través del servidor Astro.

9. Comunicación entre Servicios

La comunicación entre microservicios es síncrona vía HTTP, usando `WebClient` (reactivo) con balanceo de carga a través de Eureka. El patrón adoptado es `Service-to-Service` directo por registro en Eureka, sin bus de mensajes asíncronos en esta versión.

Mapa de dependencias entre servicios

Dependencia	Mecanismo / Endpoint
autoridades-service → UserService	PATCH /api/users/cambiar-tipo/{email} vía WebClient balanceado (lb://USER-SERVICE).
report-service → location_service	GET /api/ubicacion/{municipio} vía WebClient balanceado (lb://UBICACION).
api-gateway → todos	Enrutamiento vía Spring Cloud Gateway con lb:// load balancing.
reporta-app → api-gateway	Proxy en Astro middleware: todo /api/* → http://api-gateway:8080.
location_service → Google Maps API	REST HTTP externo vía google-maps-services-java (GeocodingApi).
UserService → Gmail SMTP	JavaMailSender vía TLS 587 para verificación de cuenta.

10. Dockerización y Despliegue

Todo el sistema se despliega con docker compose up --build desde la raíz del repositorio. Cada servicio tiene su propio Dockerfile multi-stage que minimiza la imagen final.

Patrón Dockerfile (servicios Java)

```
# Stage 1: Build
FROM maven:4.0.0-rc-5-sapmachine-25 AS builder
WORKDIR /app
COPY pom.xml . && RUN mvn dependency:go-offline
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Run (imagen mínima)
FROM eclipse-temurin:25-jdk-alpine
COPY --from=builder /app/target/*.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Orden de arranque (depends_on + healthchecks)

Fase	Condición
1. Todas las DBs	mysql/postgres con mysqladmin ping / pg_isready
2. eureka-server	wget health check sobre http://localhost:8761
3. Todos los servicios	depends_on sus DBs respectivas
4. api-gateway	depends_on: eureka-server (condition: service_healthy)
5. frontend	Sin depends_on (se conecta vía api-gateway)

11. Flujos Principales de Usuario

Flujo 1: Registro y verificación de cuenta

19. Usuario accede a /register y completa el formulario (nombre, email, password).
20. POST /api/users → UserService: valida email único, hashea password con BCrypt, genera UUID token.
21. Se envía correo con enlace de verificación vía Gmail SMTP.
22. Usuario hace clic en el enlace: GET /api/users/verify?token=UUID.
23. UserService activa la cuenta (activo=true) y borra el token.
24. Redirección a /login con mensaje de éxito.

Flujo 2: Login y acceso al sistema

25. Usuario accede a /login y envía credenciales.
26. POST /api/auth/login → UserService: valida credenciales y estado activo.
27. Respuesta: { token: "eyJ..." }. Astro almacena el token en cookie "jwt".
28. decodeJwt() lee el payload para conocer el rol (tipoUser).
29. Redirección según rol: /reports (usuario) o /admin/reports (autoridad).

Flujo 3: Crear reporte ciudadano

30. Usuario en /reports/create hace clic en el mapa Leaflet para seleccionar punto.
31. Al enviar el formulario: POST /api/ubicacion/reverse?lat=&lng= → location_service.
32. location_service llama a Google Maps API, persiste la ubicación y retorna { id, municipio, ... }.
33. Con el idLocation obtenido, se hace POST /api/report con FormData (report JSON + imagen).
34. report-service guarda el reporte en MySQL con estado PENDIENTE.
35. Redirección a /reports con mensaje de éxito.

Flujo 4: Gestión de reporte (autoridad)

36. Autoridad accede a /admin/reports con su cuenta (rol autoridad).
37. Al cargar, GET /api/autoridad/usuario/{id} recupera el municipio asignado.
38. Automáticamente filtra: GET /api/report/municipio/{municipio}.
39. report-service llama internamente a GET /api/ubicacion/{municipio}.
40. Autoridad cambia estado + observaciones vía PUT /api/report/actualizar/estatus.

12. Endpoints API — Referencia Rápida

UserService (/api/users + /api/auth)

Endpoint	Acceso	Descripción
POST /api/users	Público	Registrar nuevo usuario.
GET /api/users/verify?token=	Público	Verificar cuenta por token de correo.
PATCH /api/users/cambiar-tipo/{email}	Interno	Cambiar rol usuario↔autoridad.
POST /api/auth/login	Público	Login; retorna { token }.

autoridades-service (/api/autoridad)

Endpoint	Acceso	Descripción
POST /api/autoridad	[JWT]	Crear autoridad (cambia rol del usuario).
PUT /api/autoridad/{id}	[JWT]	Actualizar teléfono y municipio.
GET /api/autoridad/usuario/{id}	[JWT]	Detalle de autoridad por ID de usuario.

report-service (/api/report)

Endpoint	Acceso	Descripción
POST /api/report	[JWT] multipart	Crear reporte + imagen.
GET /api/report/{id}	[JWT]	Detalle de reporte.
GET /api/report/usuarios/{id}	[JWT]	Reportes del usuario.
GET /api/report/municipio/{municipio}	[JWT]	Reportes por municipio.
PUT /api/report/{id}	[JWT] multipart	Actualizar reporte.
PUT /api/report/actualizar/estatus	[JWT] autoridad	Cambiar estado.
DELETE /api/report/{id}	[JWT]	Eliminar reporte.
GET /api/report/images/{id}	Público	Servir imagen.

location_service (/api/ubicacion)

Endpoint	Acceso	Descripción
POST /api/ubicacion/reverse?lat=&lng=	Interno	Geocodificación inversa → dirección + ID.

Endpoint	Acceso	Descripción
GET /api/ubicacion/forward?query=	Interno	Geocodificación directa → coordenadas.
GET /api/ubicacion/{municipio}	[JWT]	Lista de IDs de ubicaciones en municipio.
GET /api/ubicacion/id/{id}	[JWT]	Detalle completo de ubicación.

13. Variables de Entorno y Configuración

Variable	Notas importantes
jwt.secret	Clave HMAC-SHA256 en Base64. DEBE ser idéntica en UserService y report-service.
GOOGLE_MAPS_API_KEY	Clave de Google Cloud Platform con Geocoding API habilitada.
spring.mail.password	App Password de Google (requiere 2FA activo en la cuenta de Gmail).
spring.mail.username	Cuenta de Gmail emisora de correos de verificación.
SPRING_DATASOURCE_URL	URL JDBC apuntando al contenedor DB correcto.
file.uploads-dir	Directorio de almacenamiento de imágenes. En Docker es volumen persistente.
SPRING_APPLICATION_NAME	Nombre con que se registra en Eureka.
EUREKA_CLIENT_SERVICE_URL_DEFAULTZONE	http://eureka-server:8761/eureka/

14. Guía de Instalación y Ejecución

Requisitos previos

- Docker Desktop (o Docker Engine + Compose plugin) instalado y en ejecución.
- Clave de Google Maps API con Geocoding API habilitada.
- Cuenta de Gmail con "App Password" generada.
- Git para clonar el repositorio.

Pasos de instalación con Docker

41. Clonar el repositorio y acceder al directorio:

```
git clone <URL-del-repositorio>
cd reporta
```


42. Editar las variables sensibles en docker-compose.yml:

```
GOOGLE_MAPS_API_KEY: <tu-clave-de-google-maps>
SPRING_MAIL_USERNAME: tu@gmail.com
SPRING_MAIL_PASSWORD: <app-password>
```

43. Construir e iniciar todos los servicios:

```
docker compose up --build
```

44. Esperar a que todos los contenedores estén healthy (aprox. 2-3 min).

45. Acceder a la aplicación en: http://localhost:4321

15. Repositorio del Proyecto

El código fuente completo del sistema ReportA se encuentra disponible en GitHub. El repositorio contiene todos los microservicios, el frontend, los Dockerfiles y el archivo docker-compose.yml para despliegue completo.

□ **Repositorio GitHub:** <https://github.com/reporta-project/reporta>

□ Integrantes del equipo de desarrollo

Nombre	Rol / Institución
Corona Preciado Fernando	Desarrollo · Centro Universitario de Tonalá
García de la Cruz Jesús Axel	Desarrollo · Centro Universitario de Tonalá
Fierros García Carlos Samuel	Desarrollo · Centro Universitario de Tonalá
Rodríguez Armas Juan Ramón	Desarrollo · Centro Universitario de Tonalá

16. Glosario de Términos

Término	Definición
Microservicio	Servicio autónomo con responsabilidad única, base de datos propia y desplegable de forma independiente.
Eureka	Servicio de registro y descubrimiento de Netflix OSS.
API Gateway	Punto de entrada único que enruta peticiones a los microservicios correspondientes.
JWT	JSON Web Token. Token firmado que contiene identidad y rol del usuario. Válido 8 horas.

Término	Definición
BCrypt	Algoritmo de hashing adaptativo para almacenar contraseñas de forma segura.
WebClient	Cliente HTTP reactivo de Spring WebFlux para comunicación entre servicios.
lb://	Prefijo de URI que activa el load balancing vía Eureka en Spring Cloud Gateway.
SSR	Server-Side Rendering. El HTML se genera en el servidor antes de enviarlo al cliente.
Geocodificación	Proceso de convertir coordenadas GPS en dirección legible (inversa) o viceversa (directa).
init.sql	Script SQL ejecutado al crear el contenedor de base de datos para cargar datos iniciales.
HMAC-SHA256	Algoritmo de firma criptográfica usado para firmar los tokens JWT.
Docker Compose	Herramienta para definir y ejecutar aplicaciones multi-contenedor.

ReportA — Sistema de Reportes Ciudadanos

Centro Universitario de Tonalá · Jalisco, México · 2026